

1 PhysKit 0.1.0

PhysKit é uma biblioteca Typst para construir diagramas de Física a partir de objetos, vínculos e conexões. Esta versão inicial implementa as fundações do projeto e um primeiro módulo de mecânica.

1.1 Arquitetura

A API possui dois níveis arquiteturais:

- primitivos: geometria e desenho de baixo nível;
- módulos de alto nível, atualmente `mechanics` e `vectors`.

Internamente, um núcleo geométrico compartilhado administra referenciais locais, transformações, portas direcionais e pontos de tangência. Ele não precisa ser importado em documentos comuns.

Importe apenas os módulos de alto nível necessários ao documento:

```
#import "@preview/physkit:0.1.0": mechanics, vectors
```

1.2 Objetos

Objetos são descrições imutáveis. Eles não desenharam imediatamente.

```
#let block = mechanics.box("block", label: [$m$])
#let ramp = mechanics.inclined-plane(
  "ramp",
  origin: (0, 0),
  length: 5,
  angle: 30deg,
)
```

1.3 Vínculos

Um vínculo determina a posição de um objeto em relação a outro:

```
#let support = mechanics.on-surface(
  block,
  ramp,
  distance: 3,
)
```

O centro e a orientação do bloco são resolvidos automaticamente.

Vínculos também podem acoplar vários objetos. O exemplo abaixo posiciona a polia e desloca verticalmente o teto até que o trecho de entrada da corda seja paralelo ao plano:

```
#let alignment = mechanics.align-rope-parallel(
  from: mechanics.connect(block, "right"),
  pulley: pulley,
  parallel-to: ramp,
  support: ceiling,
  support-position: 72%,
  support-distance: 0.8,
  wrap-side: "upper",
)
```

`support-position` fixa a coordenada horizontal do apoio no teto. A altura do teto, a posição da polia e o ponto tangente de entrada são calculados em conjunto. Por isso, esse recurso pertence à camada de alto nível de `mechanics`; não é necessário expor uma terceira camada pública.

Uma massa suspensa também deve ser posicionada por um vínculo, e não por uma coordenada manual:

```
#let suspension = mechanics.suspended-from(
  mass,
  pulley,
  side: "right",
  length: 1.6,
)
```

Nesse caso, `length` é o comprimento do trecho reto entre o ponto de tangência da polia e a âncora superior da massa.

1.4 Conexões

Cordas são caminhos formados por âncoras e passagens por polias:

```
#let rope = mechanics.rope(  
  "rope",  
  (  
    mechanics.connect(block, "right"),  
    mechanics.wrap(pulley, side: "upper"),  
    mechanics.connect(mass, "top"),  
  ),  
)
```

O ponto de contato de entrada é calculado como uma tangência ao círculo. Quando o corpo seguinte foi posicionado por `suspended-from`, a saída da corda utiliza a mesma porta direcional, garantindo alinhamento vertical.

1.5 Posicionamento dos rótulos

Objetos, cordas e forças aceitam `label-offset`, expresso no referencial global do diagrama. Isso permite evitar sobreposições sem alterar a geometria física:

```
#let block = mechanics.box(  
  "block",  
  label: [$m_1$],  
  label-offset: (0, 0.08),  
)
```

```
#let rope = mechanics.rope(  
  "rope",  
  path,  
  label: [$T$],  
  label-position: 48%,  
  label-offset: (0.28, 0),  
)
```

```
#let weight = mechanics.weight(  
  "weight",  
  block,  
  label: [$P_1$],  
  label-position: 55%,  
  label-offset: (0.28, 0),  
)
```

`label-position` é uma porcentagem ao longo do trecho reto rotulado. Valores de `label-offset` positivos deslocam o conteúdo para a direita e para cima. Objetos também aceitam `label-anchor` para controlar qual ponto da caixa tipográfica fica preso à coordenada.

1.6 Diagramas de corpo livre

`mechanics.free-body` isola um corpo e renderiza somente as forças associadas a ele. O corpo não precisa receber uma coordenada previamente:

```
#let body = mechanics.box("body", label: [$m$])  
#let forces = (  
  mechanics.force("weight", body, (0, -1),  
    anchor: "bottom", label: [$P$]),  
  mechanics.force("normal", body, (0, 1),  
    anchor: "top", label: [$N$]),  
  mechanics.force("friction", body, (-1, 0),  
    anchor: "left", label: [$f$]),  
)
```

```
#mechanics.free-body(body, forces: forces)
```

As âncoras top, bottom, left e right evitam que os vetores atravessem o corpo ou seu rótulo.

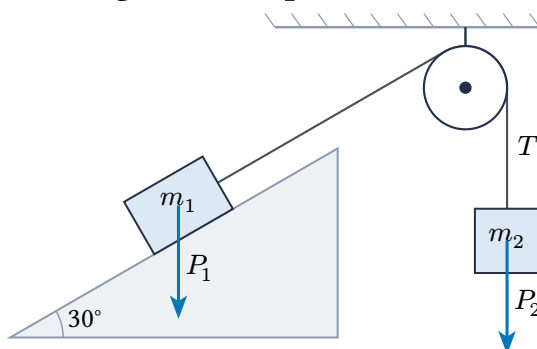
1.7 Vetores no plano cartesiano

O módulo vectors descreve vetores por componentes cartesianas ou por módulo e ângulo:

```
#let a = vectors.vector(  
  "a", (3, 2),  
  label: [$arrow(a)$],  
  show-components: true,  
)  
#let b = vectors.polar-vector(  
  "b", 2, 90deg,  
  from: a.end,  
  label: [$arrow(b)$],  
)  
#let r = vectors.resultant(  
  "r", (a, b),  
  label: [$arrow(R)$],  
)  
  
#vectors.diagram(  
  vectors: (a, b, r),  
  x-range: (-1, 5),  
  y-range: (-1, 6),  
)
```

O plano pode controlar grid, grid-step, axes, tick-step, tick-labels, x-label, y-label e origin-label. A resultante soma automaticamente as componentes dos vetores fornecidos; os pontos inicial e final permanecem disponíveis em from e end.

1.8 Diagrama completo



1.9 Vetores e corpo livre

O exemplo completo combina os dois módulos em uma única página e pode ser compilado diretamente a partir de `examples/vectors-free-body.typ`.

1.10 Estabilidade da API

A série 0.x é experimental. Mudanças incompatíveis podem ocorrer entre versões menores. A partir da versão 1.0.0, a API pública seguirá versionamento semântico estrito.